Technical Audit of Large Language Diffusion with Masking (LLaDA) Architectures: From 8B SCRATCH-Training to 100B Frontiers

The architectural evolution of large language models (LLMs) has reached a critical juncture where the long-standing dominance of autoregressive models (ARMs) is being systematically challenged by discrete diffusion paradigms. The Large Language Diffusion with Masking (LLaDA) framework, spanning from its initial 8B implementation to the 100B parameter LLaDA 2.0 series, introduces a generative methodology that prioritizes bidirectional context and iterative refinement over sequential next-token prediction.[1] This technical audit evaluates the structural, mathematical, and operational dimensions of the LLaDA architecture, identifying the unique capabilities and the persistent bottlenecks that define this emerging alternative to the transformer-based causal status quo.

Mathematical Foundation: Probabilistic Modeling and Likelihood Bounds

The fundamental distinction between LLaDA and traditional Causal Language Models (CLMs) resides in the formulation of the sequence probability distribution. While CLMs decompose the joint probability of a sequence into a product of conditional probabilities following a fixed left-to-right order, LLaDA models distributions through a non-causal forward data masking process and a learned reverse generation process.[3] This approach allows the model to leverage a full bidirectional context at every stage of the generation process, fundamentally altering the training objective and the resulting model behavior.[5]

Contrastive Analysis of Likelihood Objectives

Traditional autoregressive models are typically trained by maximizing the exact log-likelihood of the training data through a cross-entropy loss, which directly penalizes the deviation of the model's predicted next-token distribution from the ground truth. In contrast, Masked Diffusion Models (MDMs) like LLaDA optimize a variational Evidence Lower Bound (ELBO) of the log-likelihood.[7] The forward process in LLaDA involves the progressive corruption of data by independently masking tokens in an original sequence x_0 with a probability $t \in $. At any given time t, the sequence x_t is partially masked, with each token being masked with

probability t or remaining unmasked with probability $1 - t$.[3]

The reverse process, which constitutes the actual generative capability of the model, is parameterized by a Transformer-based mask predictor $p_\theta(\cdot|x_t)$ that takes a partially masked sequence as input and predicts all masked tokens simultaneously.[9] The LLaDA training objective uses a cross-entropy loss computed only on the masked tokens, weighted by the inverse of the corruption level $1/t$:

\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, x_t \sim q(x_t|x_0)} \left{} -\log p_\theta(x_0^{(i)}|x_t) \right]

The theoretical justification for this objective is that it serves as a principled upper bound on the negative log-likelihood (NLL) of the model distribution.[4] This ensures that as the model minimizes the weighted cross-entropy loss, it is effectively approximating the true language distribution. Unlike standard Masked Language Models (MLMs) such as BERT, which utilize a fixed masking ratio (e.g., 15%), LLaDA samples t uniformly from $$. This subtle but significant difference allows LLaDA to function as a generative model by learning to restore sequences from arbitrary noise levels, including the fully masked state at $t = 1$.[7]

The Remasking Confidence Strategy

A critical component of LLaDA's generative efficacy is the 'remasking' strategy employed during the reverse diffusion process. During inference, the model does not attempt to generate a coherent sequence in a single forward pass; instead, it discretizes the reverse process into multiple steps.[11] At each intermediate step, the mask predictor suggests possible tokens for all masked positions. However, rather than committing to all predicted tokens immediately, the model adopts a low-confidence remasking strategy.[11]

In this mechanism, a subset of the predicted tokens—specifically those with the lowest prediction confidence or highest entropy—is re-masked before the next iteration. This allows the model to iteratively refine its output, treating the high-confidence predictions as structural anchors that inform the subsequent restoration of more uncertain tokens.[13] This "survival of the fittest" approach effectively mitigates the accumulation of errors that often plagues non-autoregressive models, as the model can "change its mind" about a token if the surrounding context becomes more defined in later steps.[14]

Optimization Metric	Causal Language Model (CLM)	Masked Diffusion Model (MDM)
Probability	Chain rule of probability	Joint distribution via ELBO

Decomposition		
Training Objective	Exact Cross-Entropy	Upper bound on NLL with $1/t$ term
Masking Distribution	Static (N/A)	Dynamic $t \sim$
Inference Path	Sequential (Greedy/Beam)	Iterative Refinement (Remasking)
Context Accessibility	Unidirectional (Past only)	Bidirectional (Global)

Benchmark Dominance: Solving the Reversal Curse

The bidirectional nature of LLaDA provides a structural advantage over autoregressive models in tasks requiring non-linear logic and symmetrical reasoning. The most prominent example of this is the "Reversal Curse," a phenomenon where traditional LLMs trained on data indicating that "A is B" often fail to recognize that "B is A" unless explicitly trained on the inverted statement.[16]

Reversal Poem Completion Metrics

In empirical evaluations using the Reversal Poem Completion benchmark, LLaDA variants demonstrate near-perfect bidirectional understanding, significantly outperforming market-leading models like GPT-4o and Llama 3.[9] While AR models show a severe performance drop when required to complete a poem from the end toward the beginning, LLaDA maintains consistent accuracy because its full-attention backbone treats all positions as globally accessible during the denoising process.[14]

Benchmark Task	Llama 3 8B Accuracy	GPT-4o Accuracy	LLaDA 8B Accuracy
Forward Poem Completion	74%	87%	91%
Reversal Poem Completion	51.8%	34.3%	91%

Performance Delta (Gap)	22.2%	52.7%	0.0%

The bidirectional nature specifically solves the logic gap because it removes the inductive bias toward left-to-right temporal flow. In an AR model, the embedding for token B at training time is only updated based on the hidden states of A. If the model is asked to predict A given B at test time, the internal attention weights for B have never been optimized to attend to the "future" position where A would reside.[15] LLaDA avoids this by treating tokens uniformly; at any given training step, any token can serve as the context for any other token, regardless of their relative sequence position. This makes the model inherently capable of biographical inversions, mathematical proof reversals, and complex logical "what-if" scenarios.[7]

Domain-Specific Performance Wins

The architectural flexibility of LLaDA extends to structured data generation, particularly in code infilling and mathematical reasoning. Because the model views text as a canvas to be filled rather than a stream to be extended, it excels at "Fill-in-the-Middle" (FIM) tasks. In code completion, LLaDA can reconcile the logic of a function by simultaneously looking at the imports defined above and the function calls made below the cursor, a capability that AR models must simulate via specialized (and often inefficient) pre-training techniques.[18]

In mathematical reasoning, LLaDA 8B has demonstrated a score of 70.7% on the GSM8K benchmark, compared to Llama 3 8B's 53.1%. This performance gap is a second-order effect of the remasking strategy: during the multiple denoising passes, the model can "check its work." If an intermediate step in a multi-stage math derivation is inconsistent with the final result, the bidirectional attention allows the model to correct the logic in earlier positions during later steps of the diffusion trajectory.[7]

Efficiency Paradox: Throughput vs Complexity

A central irony of the current LLaDA implementation is that despite being a non-autoregressive model—which are theoretically designed for massive parallelization—it often exhibits lower inference throughput than KV-cached autoregressive models for long-form generation. This is referred to as the "Efficiency Paradox," driven by the diverging computational characteristics of sequential and iterative generation.[21]

FLOPs-per-token Ratio and Memory Bandwidth

Autoregressive models are fundamentally memory-bound during the decoding phase. By utilizing a Key-Value (KV) Cache, they avoid recomputing the hidden states of previous tokens,

meaning each new token generation only requires a single forward pass over the newest token's embedding.[23] The computational cost grows linearly with sequence length in practice. Diffusion models, however, are typically compute-bound. Because LLaDA relies on full bidirectional attention, it must process the entire sequence (both already-unmasked and still-masked tokens) at every denoising step to maintain global coherence.[25]

For a generation task of length L with T denoising steps, the theoretical complexity gap is stark:

- **AR Model:** $O(L \cdot d^2)$ total FLOPs where hidden states are cached.
- **LLaDA:** $O(T \cdot L \cdot d^2 + T \cdot L^2 \cdot d)$ total FLOPs.[24]

As L increases, the quadratic L^2 term in the LLaDA attention mechanism becomes the dominant bottleneck. Even if T (the number of steps) is relatively small (e.g., 64 steps), the necessity of re-processing every token at every step means that for long outputs (e.g., 2000 tokens), LLaDA performs far more total floating-point operations per generated token than an AR model. This explains why AR models can achieve 13.7x higher throughput than LLaDA-Instruct-8B in certain long-document benchmarks.[22]

The MoE Efficiency Bottleneck

The integration of Mixture-of-Experts (MoE) in LLaDA 2.0 further complicates the efficiency profile. In LLaDA 2.0-flash (100B), only a subset of experts is active for any given token (e.g., 6.1B active parameters).[28] While this reduces the active FLOPs per forward pass, it introduces an "expert explosion" during parallel decoding. In AR decoding, only 1-2 experts are loaded from HBM to SRAM per layer per step because only one token is being processed. In parallel diffusion, N tokens are processed simultaneously. If these N tokens independently select different experts, the system must load nearly all unique experts from memory in every layer, every step.[21] This negates the memory savings of MoE and can result in inference speeds slower than a dense model of significantly smaller scale.[21]

Throughput Metric	**AR Model (Llama 3)**	**LLaDA 8B (v1)**	**LLaDA 2.0-flash (100B)**
Token Generation Style	Sequential (1/step)	Parallel (Blocks)	Parallel (Blocks)
Hardware	Memory Bandwidth	Compute (FLOPs)	HBM-to-SRAM

Bottleneck			Traffic
Context Length Scaling	Linear ($O(L)$)	Quadratic ($O(L^2)$)	Quadratic ($O(L^2)$)
Inference Efficiency	High (KV-Cache)	Low (Recomputation)	Moderate (dInfer Engine)

Technical Regions of Failure: Identifying Structural Weaknesses

The LLaDA architecture is susceptible to specific failure modes that arise from its reliance on discrete denoising and full bidirectional context. These weaknesses define the "technical regions of failure" where the model's reliability decays.

Quadratic Complexity and the Context Length Wall

LLaDA is built on a vanilla Transformer backbone which, while powerful for representation, lacks the "sequential shortcut" provided by the causal masking and KV-caching of AR models. As the sequence length grows, the computational cost of re-calculating the full attention matrix at every step becomes prohibitive.[30] At context lengths above 32k tokens, the latency for a single response becomes impractical for real-time applications unless specialized optimizations are used.[32]

Sparse attention methods like SparseD have been proposed to mitigate this by pre-computing head-specific sparse patterns in early denoising steps and reusing them, achieving up to 1.5x speedup at 64k context.[33] However, without these techniques, the quadratic bottleneck acts as a hard ceiling for the scalability of standard LLaDA models in long-horizon tasks such as legal document review or multi-hour video transcript analysis.[34]

Coherence Decay and Mask-Ratio Sensitivity

A unique failure mode of LLaDA is the decay of global semantic coherence when the generation length deviates from the pre-training mask-ratio distribution. The model's internal understanding of the "correct" sequence of unmasking is learned through a random masking schedule where $t \sim U$. If the requested generation length is significantly longer than the documents used during fine-tuning, the model struggles to maintain a consistent global structure.[11] This often results in local coherence (sentences that look fine in isolation) but global contradiction (the end of the text forgetting the premises set at the beginning).

Furthermore, LLaDA is highly sensitive to sampling temperature. While AR models can use high temperatures to encourage creativity, LLaDA can fall into "local optima" where it becomes stuck repeating a high-confidence but incorrect phrase across an entire block.[2] Because tokens are generated in parallel, a single confident hallucination can propagate "sideways" through the attention mechanism, forcing all other tokens in the block to align with the error in a self-reinforcing loop of inaccuracy.[15]

The Permanent Token Problem

In standard absorbing-state diffusion models like LLaDA 8B, once a token is replaced by a real token, it is "locked in" for all subsequent steps.[36] This creates "context rigidity": if the model makes a suboptimal choice at step 10 of a 64-step diffusion run (when context is still sparse), that error becomes a permanent constraint for the remaining 54 steps.[38] This is the primary driver of n-gram repetitions and syntactical glitches in the v1.0 models. LLaDA 2.1 attempts to solve this via Token-to-Token (T2T) editing, allowing the model to revisit and correct previously unmasked tokens.[40]

LLaDA 2.0 Training: The Warmup-Stable-Decay Paradigm

Scaling diffusion models to 100B parameters required a departure from the scratch-training methods used for the 8B variant. LLaDA 2.0 establishes a new paradigm for "continual pre-training" (CPT) that inherits knowledge from existing, mature AR checkpoints.[2]

The WSD 3-Phase Training Scheme

The systematic conversion process utilizes a novel strategy based on block diffusion, which allows the model to progressively adapt to bidirectional patterns while maintaining its underlying linguistic world model.[2]

1. **Warmup Phase:** The model begins training as a Block Diffusion Language Model (BDLM). The block size is progressively increased (e.g., from 1 to 4096 tokens). This phase allows the causal attention weights to "unfreeze" and begin capturing dependencies that cross the causal boundary.[43]
2. **Stable Phase:** Once the model can handle full-sequence context (4096 tokens), it enters the full MDLM regime. It is trained on large-scale corpora to stabilize its understanding of high-noise denoising.[2]
3. **Decay Phase:** To regain inference efficiency, the model is converted back into a BDLM with a compact block size (e.g., 32). This step distills the global context knowledge into a structure that can support KV-cache reuse and faster variable-length generation.[43]

Training Phase	Mechanism	Context Pattern	Primary Objective

Warmup	Incremental block growth	Local Bidirectional	Architectural adaptation from AR
Stable	Full-sequence diffusion	Global Bidirectional	Deep denoising & knowledge acquisition
Decay	Block size reduction	Hybrid (AR + Local Diff)	Deployment optimization (KV-cache)

Confidence-Aware Parallel (CAP) Training

To maximize the advantages of parallel decoding, LLaDA 2.0-flash-CAP introduces an auxiliary confidence prediction loss during training. Standard diffusion training only ensures token correctness, which often leaves the model uncertain about its own predictions.[37] CAP training explicitly optimizes the model to be "sharper," minimizing entropy for correct predictions.[10] This enables more aggressive parallel generation at inference time, as the system can unmask a higher percentage of tokens per step without a significant drop in quality.[10] In benchmarks, CAP variants achieve 1.46x faster generation than base LLaDA 2.0 models with only a negligible (1.72%) performance trade-off.[37]

Alignment and Preference Optimization: Navigating Intractability

Aligning diffusion models with human preferences represents one of the most significant research hurdles in the field. Mature pipelines like PPO (Proximal Policy Optimization) or DPO (Direct Preference Optimization) used for AR models rely heavily on sequence-level log-likelihoods, which are intractable for diffusion models due to their iterative, multi-step nature.[8]

Direct Preference Optimization Limitations

In AR models, DPO can directly optimize the policy by comparing the log-likelihood of preferred versus dispreferred responses. In LLaDA, there is no single-step likelihood for a complete sentence; the probability depends on the entire sampled trajectory of the diffusion process.[8] Applying standard DPO results in high-variance gradients that can lead to policy collapse or "robotic" outputs that follow instructions literally but lack nuanced persona constraints.[11]

Emerging Alignment Frameworks

To address these limitations, several specialized frameworks have been developed for dLLMs:

- **Variance-Reduced Preference Optimization (VRPO):** Utilized for LLaDA 1.5, this framework analyzes the bias and variance of ELBO estimators. It introduces unbiased variance reduction strategies like antithetic sampling and optimal budget allocation across diffusion timesteps, resulting in consistent gains in mathematical and reasoning benchmarks.[47]
- **ELBO-based Block-level Policy Optimization (EBPO):** This is the first large-scale RL framework tailored for 100B-scale diffusion models. It uses "Vectorized Likelihood Estimation" to perform parallelized bound computation across sequences, allowing the model to align its generation trajectories with human intent without marginalizing over every possible path.[38]
- **ELBO-KTO:** An unpaired preference optimization framework that allows LLaDA to learn from binary "good/bad" feedback signals rather than requiring costly paired comparisons. This formulation enjoys bounded bias and has demonstrated significant win-rate improvements on general dialogue tasks.[45]

Cost-Effectiveness and Total Cost of Ownership (TCO)

Providing a TCO comparison for serving LLaDA versus quantized AR models requires analyzing the intersection of hardware requirements, batching efficiency, and inference engine optimizations.

Serving LLaDA 100B on H800 Clusters

The serving footprint for LLaDA 2.0-flash (100B) is significant. At half-precision (FP16/BF16), the model weights alone require approximately 200GB of VRAM. An 8-GPU H800 cluster node with 80GB VRAM per GPU provides 640GB of total capacity, which is sufficient to host the model and its associated dual-KV caches.[42]

Unlike AR models where the KV cache grows incrementally and only requires appending new vectors, LLaDA requires global cache updates or high-frequency refreshes at every denoising step.[42] This puts a much higher load on the NVLink/intra-node interconnect bandwidth. However, the dInfer framework mitigates this by modularizing the iteration manager and supporting block-level parallel decoding.[42]

Metric	AR Model (quantized 405B)	LLaDA 2.0-flash (100B)
GPU Requirement	8x H100 (80GB)	8x H800 (80GB)

Active Parameters	405B	6.1B (MoE)
Peak Throughput	~50 - 150 TPS	~535 - 892 TPS (CAP/T2T)
Serving Framework	vLLM / TensorRT-LLM	dInfer / SGLang
Quantization Support	Mature (GPTQ/AWQ)	Emerging (Block FP8)

TCO Comparison: H800 Serving vs. Quantized AR

From a TCO perspective, LLaDA 2.0 is more cost-effective for batch processing and high-throughput reasoning tasks but less efficient for low-latency, single-user chat.

- **Cloud Instance Costs:** Running an H100/H800 node costs approximately $77–$98 per hour depending on reservation status.[52]
- **Cost per Token:** Because LLaDA 2.0-flash activated parameters (6.1B) are significantly lower than high-end AR models (like Llama 3.1 405B), its raw compute cost per forward pass is lower. However, its multi-step generation means it must perform ~32–64 passes to finish a request, whereas an AR model performs one pass per token.[24]
- **The Breakeven Point:** LLaDA becomes more cost-effective when the "Tokens Per Forward" (TPF) exceeds the number of diffusion steps. For example, LLaDA 2.1 has reached a TPF of 13.8 in speedy mode. If generating 1000 tokens, it requires only ~72 forward passes. An AR model requires 1000 passes. In this regime, LLaDA's TCO on an H800 cluster is significantly lower than serving a non-distilled AR model.[37]

Technical Analysis Cheat Sheet (2025–2026)

Based on the current technical trajectory, the following summary identifies the strategic position of LLaDA architectures in the modern LLM landscape.

Where it Tops the Field (Successes)

- **The Reversal Reasoning Leader:** LLaDA is the world leader in solving the reversal curse. Its bidirectional context modeling is near-perfect, outperforming GPT-4o by over 50 percentage points in reverse-completion accuracy.[9]
- **Mathematical Precision:** LLaDA 8B variants hit 70.7% on GSM8K, outclassing AR models of similar size. This stems from its ability to revise its internal logic states during the multiple denoising passes.[7]
- **Fill-in-the-Middle (FIM):** By treating text as a holistic canvas, LLaDA is the superior choice for code infilling and editing. It reconciles dependencies above and below the cursor simultaneously without the need for specialized training tricks.[15]

- **Data Efficiency:** LLaDA 8B achieved performance parity with Llama 3 8B while using only 2.3 trillion training tokens, compared to Llama's 15 trillion tokens, indicating a significant data-potential advantage.[7]

Where it Lacks (Weaknesses)

- **Latency for Short-form Generation:** For small outputs (e.g., "Yes" or "No"), LLaDA is inefficient because it still requires 25–50 denoising steps, whereas an AR model can finish in a single pass.[11]
- **Creative Polish:** It lacks the extensive preference alignment of models like Claude or GPT-4o. It can feel more "robotic" or struggle with extremely nuanced stylistic constraints.[5]
- **Inference Costs for Long Prose:** For 2000-word creative writing, the quadratic L^2 complexity and re-processing of the entire sequence at every step make it less cost-effective than speculative-decoded AR models.[22]

Technical Region of Failure (The Failure Points)

- **The Quadratic Bottleneck:** LLaDA uses a vanilla Transformer backbone. At context lengths exceeding 32k, the computational cost grows by N^2, leading to an efficiency wall unless UltraLLaDA optimizations are implemented.[32]
- **Local Optima and Hallucination:** If the sampling temperature isn't perfectly tuned ($T = 0.0$ is the stable default), the model can get stuck in a self-reinforcing loop of incorrect tokens that dominate a parallel-generated block.[2]
- **Context Rigidity:** In absorbing-state models (v1.0), tokens unmasked in early steps cannot be revised later. This leads to persistent n-gram repetitions that self-correction only partially fixes in later stages.[38]

Nuanced Conclusions and Outlook

The technical audit of the LLaDA architecture family confirms that diffusion models have transitioned from a "niche academic route" to a viable, frontier-scale alternative for large language modeling. By scaling to 100B parameters through systematic AR-to-MDLM conversion, LLaDA 2.0 has successfully explored a path to overcoming the sequential generation bottleneck that has constrained the field for a decade.[41]

The architectural dominance of LLaDA in non-linear reasoning, particularly its immunity to the reversal curse and its data efficiency, makes it a critical tool for structured domains like mathematics, software engineering, and logical planning. The bidirectional nature of its attention mechanism provides a "global view" of language that autoregressive models can only

mimic through excessive data exposure.[16]

However, the practical deployment of LLaDA is currently bounded by the "Efficiency Paradox." While theoretically faster due to parallel decoding, the high FLOPs-per-token ratio driven by quadratic complexity and expert explosion in MoE variants limits its utility in general-purpose, long-form conversational AI. The roadmap forward, defined by the transition to LLaDA 2.1 and the introduction of T2T editing, suggests that the field is moving toward hybrid architectures that combine the bidirectional planning of diffusion with the inference-time efficiency of autoregression.[40]

For organizations operating high-performance H800 clusters, LLaDA 2.0-flash-CAP offers a powerful reasoning engine that provides a 2.1x speedup over standard AR baselines when properly configured for parallel decoding. In the long term, the success of the LLaDA project signals that the future of large language models may not be solely autoregressive, but rather a sophisticated synergy of iterative refinement and global contextual awareness.[37]

Works cited

1. [2502.09992] Large Language Diffusion Models - arXiv, accessed February 25, 2026, https://arxiv.org/abs/2502.09992
2. LLaDA2.0: Scaling Up Diffusion Language Models to 100B - arXiv, accessed February 25, 2026, https://arxiv.org/html/2512.15745v1
3. Large Language Diffusion Models - GPTrack.ai, accessed February 25, 2026, https://gptrack.ai/paper/dLLM.pdf
4. Large Language Diffusion Models - arXiv.org, accessed February 25, 2026, https://arxiv.org/pdf/2502.09992
5. LLaDA-8B-Instruct Diffusion Model, accessed February 25, 2026, https://www.emergentmind.com/topics/llada-8b-instruct
6. Paper Review: Large Language Diffusion Models - Andrey Lukyanenko, accessed February 25, 2026, https://andlukyane.com/blog/paper-review-llada
7. Diffusion-Based LLMs (dLLMs) and LLaDA | by Alican Kiraz - Medium, accessed February 25, 2026, https://alican-kiraz1.medium.com/diffusion-based-llms-dllms-and-llada-88e80aba13a5
8. LLaDA 1.5: Variance-Reduced Preference Optimization for Large Language Diffusion Models - arXiv, accessed February 25, 2026, https://arxiv.org/pdf/2505.19223
9. Large Language Diffusion Models | alphaXiv, accessed February 25, 2026, https://www.alphaxiv.org/overview/2502.09992v3
10. ML-GSAI/LLaDA: Official PyTorch implementation for "Large Language Diffusion Models" - GitHub, accessed February 25, 2026, https://github.com/ML-GSAI/LLaDA
11. Large Language Diffusion Models - arXiv, accessed February 25, 2026, https://arxiv.org/html/2502.09992
12. Large Language Diffusion Models - arXiv, accessed February 25, 2026,

https://arxiv.org/html/2502.09992v3
13. LLaDA: The Diffusion Model That Could Redefine Language Generation, accessed February 25, 2026, https://towardsdatascience.com/llada-the-diffusion-model-that-could-redefine-language-generation/
14. NeurIPS 2025 oral: Diffusion model officially enters LLM | by L.J. - Medium, accessed February 25, 2026, https://medium.com/@zljdanceholic/neurips-2025-oral-diffusion-model-officially-enters-llm-fc68441cebc0
15. Large Language Masked Diffusion Models (LLaDA) - Javier Solís García, accessed February 25, 2026, https://javiersolisgarcia.com/posts/llada/
16. LLaDA: Revolutionizing AI Text Generation & Reasoning with Diffusion-Based Innovation | by Santhosraj | Medium, accessed February 25, 2026, https://medium.com/@santhosraj14/llada-revolutionizing-ai-text-generation-reasoning-with-diffusion-based-innovation-81ed3fd5168f
17. The Bidirectional Advantage: How LLaDA's Diffusion Architecture Outthinks Traditional LLMs - Greg Robison, accessed February 25, 2026, https://gregrobison.medium.com/the-bidirectional-advantage-how-lladas-diffusion-architecture-outthinks-traditional-llms-4fa09298c94e
18. Improving Sampling for Masked Diffusion Models via Information Gain - arXiv, accessed February 25, 2026, https://arxiv.org/html/2602.18176v1
19. Thinking Inside the Mask: In-Place Prompting in Diffusion LLMs - arXiv, accessed February 25, 2026, https://arxiv.org/html/2508.10736v1
20. Can I Have Your Order? Monte-Carlo Tree Search for Slot Filling Ordering in Diffusion Language Models - arXiv, accessed February 25, 2026, https://arxiv.org/html/2602.12586v1
21. Decoupling Memory from Parallelism in Mixture-of-Experts Diffusion LLMs - arXiv.org, accessed February 25, 2026, https://www.arxiv.org/pdf/2602.00879
22. How Efficient Are Diffusion Language Models? A Critical Examination of Efficiency Evaluation Practices - arXiv, accessed February 25, 2026, https://arxiv.org/html/2510.18480v1
23. The Secret Behind Fast LLM Inference: Unlocking the KV Cache - Towards AI, accessed February 25, 2026, https://pub.towardsai.net/the-secret-behind-fast-llm-inference-unlocking-the-kv-cache-9c13140b632d
24. Optimize for inference too, not just training FLOPs - MatX, accessed February 25, 2026, https://matx.com/research/lifetime_llm_cost
25. Consistency diffusion language models: Up to 14x faster inference without sacrificing quality, accessed February 25, 2026, https://www.together.ai/blog/consistency-diffusion-language-models
26. FOCUS: DLLMs Know How to Tame Their Compute Bound - KAUST Repository, accessed February 25, 2026, https://repository.kaust.edu.sa/bitstreams/1ecea31c-326f-4669-afd8-f7f4df3415a5/download
27. How Efficient Are Diffusion Language Models? A Critical Examination of Efficiency

Evaluation Practices | alphaXiv, accessed February 25, 2026, https://www.alphaxiv.org/overview/2510.18480v1
28. inclusionAI/LLaDA2.0-flash-preview - Hugging Face, accessed February 25, 2026, https://huggingface.co/inclusionAI/LLaDA2.0-flash-preview
29. Dynamic Expert Sharing: Decoupling Memory from Parallelism in Mixture-of-Experts Diffusion LLMs - arXiv, accessed February 25, 2026, https://arxiv.org/html/2602.00879v1
30. Sparse Attention for Diffusion Language Models - arXiv.org, accessed February 25, 2026, https://arxiv.org/pdf/2509.24014
31. [Literature Review] How Efficient Are Diffusion Language Models? A Critical Examination of Efficiency Evaluation Practices - Moonlight, accessed February 25, 2026, https://www.themoonlight.io/en/review/how-efficient-are-diffusion-language-models-a-critical-examination-of-efficiency-evaluation-practices
32. UltraLLaDA: Extended Context Diffusion LLM - Emergent Mind, accessed February 25, 2026, https://www.emergentmind.com/topics/ultrallada
33. Sparse Attention for Diffusion Language Models - arXiv, accessed February 25, 2026, https://arxiv.org/html/2509.24014v1
34. SparseD: Sparse Attention for Diffusion Language Models - OpenReview, accessed February 25, 2026, https://openreview.net/forum?id=dwbrZtYP04
35. VidLaDA: Diffusion-Based Video LLM - Emergent Mind, accessed February 25, 2026, https://www.emergentmind.com/topics/vidlada
36. Relaxing Positional Alignment in Masked Diffusion Language Models - arXiv, accessed February 25, 2026, https://arxiv.org/html/2601.22947v1
37. inclusionAI/LLaDA2.0-flash-CAP - Hugging Face, accessed February 25, 2026, https://huggingface.co/inclusionAI/LLaDA2.0-flash-CAP
38. LLaDA2.1 vs Autoregressive Baselines: Is Diffusion Finally ... - Reddit, accessed February 25, 2026, https://www.reddit.com/r/LLMDevs/comments/1r162ky/llada21_vs_autoregressive_baselines_is_diffusion/
39. CoRe: Context-Robust Remasking for Diffusion Language Models - arXiv, accessed February 25, 2026, https://arxiv.org/html/2602.04096v1
40. LLaDA2.1: Speeding Up Text Diffusion via Token Editing - arXiv, accessed February 25, 2026, https://arxiv.org/pdf/2602.08676
41. Milestone: 100B Diffusion Language Model Reaches 892 Tokens/Second, Successfully Exploring an Alternative Path for AI - 36氪, accessed February 25, 2026, https://eu.36kr.com/en/p/3678264053719682
42. [2512.15745] LLaDA2.0: Scaling Up Diffusion Language Models to 100B - arXiv, accessed February 25, 2026, https://arxiv.org/abs/2512.15745
43. Milestone: First 100B Diffusion Language Model Unveiled - Technical Report Details Inside, accessed February 25, 2026, https://eu.36kr.com/en/p/3592063556468736
44. inclusionAI/LLaDA2.0-mini-CAP - Hugging Face, accessed February 25, 2026, https://huggingface.co/inclusionAI/LLaDA2.0-mini-CAP
45. Aligning Diffusion Language Models via Unpaired Preference Optimization -

ResearchGate, accessed February 25, 2026, https://www.researchgate.net/publication/397006244_Aligning_Diffusion_Language_Models_via_Unpaired_Preference_Optimization
46. Was RLHF a Detour? Rethinking LLM Alignment with DPO | by Roshan K Tiwari - Medium, accessed February 25, 2026, https://medium.com/data-science-in-your-pocket/was-rlhf-a-detour-rethinking-llm-alignment-with-dpo-0e219f3184a2
47. LLaDA 1.5: Variance-Reduced Preference Optimization for Large Language Diffusion Models - arXiv.org, accessed February 25, 2026, https://arxiv.org/html/2505.19223v2
48. LLADA 1.5: VARIANCE-REDUCED PREFERENCE OPTI- MIZATION FOR LARGE LANGUAGE DIFFUSION MODELS - OpenReview, accessed February 25, 2026, https://openreview.net/pdf?id=7dhe87Qjjq
49. Aligning Diffusion Language Models via Unpaired Preference Optimization - arXiv, accessed February 25, 2026, https://arxiv.org/html/2510.23658v2
50. Llama3-405B: How to run an extra large open source LLM on Modal, accessed February 25, 2026, https://modal.com/blog/how_to_run_llama_405b_article
51. dInfer: An Efficient Inference Framework for Diffusion Language Models - GitHub, accessed February 25, 2026, https://github.com/inclusionAI/dInfer
52. On-Premise vs Cloud: Generative AI Total Cost of Ownership (2025 Edition) - Lenovo Press, accessed February 25, 2026, https://lenovopress.lenovo.com/lp2225.pdf
53. Serving Quantized LLMs on NVIDIA H100 Tensor Core GPUs | Databricks Blog, accessed February 25, 2026, https://www.databricks.com/blog/serving-quantized-llms-nvidia-h100-tensor-core-gpus
54. Diffusion Language Models are Super Data Learners - arXiv.org, accessed February 25, 2026, https://arxiv.org/html/2511.03276v1
55. How Efficient Are Diffusion Language Models? A Critical Examination of Efficiency Evaluation Practices - arXiv, accessed February 25, 2026, https://arxiv.org/html/2510.18480v3
56. UltraLLaDA: Scaling the Context Length to 128K for Diffusion Large Language Models, accessed February 25, 2026, https://openreview.net/forum?id=68DGlhlvD9
57. Large Language Diffusion Models: A New Frontier in AI | by Sai Krishna Reddy Mudhiganti, accessed February 25, 2026, https://medium.com/@saimudhiganti/large-language-diffusion-models-a-new-frontier-in-ai-0d261a1f6757